

Arquitecturas de Software para Aplicaciones Empresariales

Validaciones con Bean Validator

PROGRAMA DE INGENIERIA DE SISTEMAS

Ing. Daniel Eduardo Paz Perafán (danielp@Unicauca.edu.co)

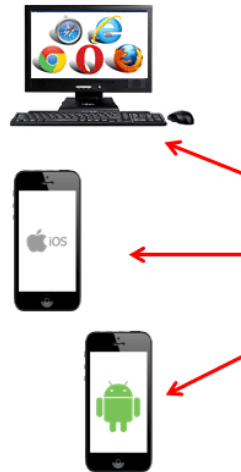
Ing. Pablo A. Magé (pmage@Unicauca.edu.co)



Donde realizar las validaciones

UNIVERSIDAD DEL CAUCA – FIET
DEPARTAMENTO DE SISTEMAS

FRONTEND



By Parzibyte

Agregar producto

Nombre del producto
 ✖
El nombre debe medir entre 1 y 50

Código de barras
 ✖
El código debe medir entre 1 y 50

Existencia actual
 ✖
Debes especificar la existencia

Precio
 ✖
Debes especificar el precio

Guardar Ver todos

© 2019 de ventas Spring Boot API con ❤️ por Parzibyte

BACKEND



BASE DE DATOS



Respuesta

	Versión	Código respuesta
Primera línea	HTTP/1.1	200 OK
Encabezados	Server: wikipedia.org Content-Type: text/html Content-Lenght: 2026	
Cuerpo	<html> ... </html>	

¿Cómo modificamos la respuesta HTTP?

- ❖ Bean Validation es el estándar de facto para implementar la lógica de validación en el ecosistema Java
- ❖ JSR 380 es una especificación de la API de Java para la validación de beans, que forma parte de Jakarta EE y JavaSE. Esto asegura que las propiedades de un bean cumplen con criterios específicos, utilizando anotaciones como @NotNull , @Min y @Max .
- ❖ Esta versión requiere Java 8 o superior y aprovecha las nuevas funciones agregadas en Java 8, como las anotaciones de tipo y la compatibilidad con nuevos tipos como Optional y LocalDate .

Según la especificación JSR 380, la dependencia validation-api contiene las API de validación estándar:

```
<dependency>  
  <groupId>javax.validation</groupId>  
  <artifactId>validation-api</artifactId>  
  <version>2.0.1.Final</version>  
</dependency>
```

Hibernate Validator es la implementación de referencia de la API de validación. Para usarlo, necesitamos agregar la siguiente dependencia:

```
<dependency>  
  <groupId>org.hibernate.validator</groupId>  
  <artifactId>hibernate-validator</artifactId>  
  <version>6.0.13.Final</version>  
</dependency>
```

hibernate-validator está completamente separado de los aspectos de persistencia de Hibernate. Entonces, al agregarlo como una dependencia, no agregamos estos aspectos de persistencia al proyecto.

La especificación Bean Validation define un conjunto de anotaciones que proporcionan una colección de restricciones genéricas y básicas y que podemos comprobar en la [documentación](#).

```
@Entity
@Table(name = "Usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @NotNull
    @Size(min = 5, max = 45)
    @Column(nullable = false, length = 45)
    private String username;

    @NotNull
    @Email
    private String email;

    @PastOrPresent
    private Date fechaRegistro;

    @Pattern(regexp = "[6][0-9]{8}")
    private String telefono;

    @PositiveOrZero
    private float salario;
}
```

```
import javax.validation.constraints.Email;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.PastOrPresent;
import javax.validation.constraints.Pattern;
import javax.validation.constraints.PositiveOrZero;
import javax.validation.constraints.Size;
```

La validación la debemos colocar en las clases DTO, hay que evitar colocarlas en las clases Entity.

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class ClienteDTO {
    private Integer id;
    @NotNull(message = "{user.name.empty}")
    @Size(min = 5, max = 45, message = "la cantidad de caracteres del nombre debe estar entre 5 y 45")
    private String nombre;
    @NotNull(message = "{user.lastname.empty}")
    @Size(min = 5, max = 45, message = "{user.lastname.length}")
    private String apellido;
    @NotNull(message = "{user.email.empty}")
    @Email(message = "{user.email.mask}")
    private String email;
    @PastOrPresent(message = "{user.date.past}")
    private Date createdAt;
    @Pattern(message = "{user.telephone.pattern}", regexp = "[6][0-9]{8}")
    private String telefono;
    @PositiveOrZero(message = "{user.salary.positive}")
    private float salario;
}
```

```
@NotNull  
@Size(min = 5, max = 45)  
@Column(nullable = false, length = 45)  
private String username;
```

El nombre es un campo obligatorio con un tamaño comprendido entre 5 y 45 caracteres. Si simplemente queremos que contenga al menos un carácter podemos utilizar [@NotBlank](#)

```
@NotNull  
@Email  
private String email;
```

El email es obligatorio y debe tener el correspondiente formato.

```
@PastOrPresent  
private Date fechaRegistro;
```

La fecha de registro debe estar en el pasado o en el presente

```
@Pattern(regexp = "[6][0-9]{8}")  
private String telefono;
```

El número de teléfono es opcional pero si se proporciona debe ser un número de 9 dígitos que empiece por seis.

```
@PositiveOrZero  
private float salario;
```

El salario debe ser un número 0 o un número positivo

Todas las anotaciones utilizadas en el ejemplo son anotaciones JSR estándar:

@NotNull valida que el valor de la propiedad anotado no es *nulo* .

@NotEmpty valida que la propiedad no sea nula ni esté vacía; se puede aplicar a valores de *cadena* , *colección* , *mapa* o *matriz* .

@NotBlank se puede aplicar solo a valores de texto y valida que la propiedad no sea nula ni carente de espacios en blanco.

@AssertTrue valida que el valor de la propiedad anotado sea *verdadero*.

@Size valida que el valor de la propiedad anotado tiene un tamaño entre los atributos *min* y *max* ; se puede aplicar a las propiedades de *cadena* , *colección* , *mapa* y *matriz*.

@Min valida que la propiedad anotada tiene un valor no menor que el atributo *value* .

@Max valida que la propiedad anotada tiene un valor no mayor que el atributo *value* .

@Email valida que la propiedad anotada es una dirección de correo electrónico válida.

Algunas anotaciones aceptan atributos adicionales, pero el atributo de *mensaje* es común a todos ellos. Este es el mensaje que normalmente se presentará cuando el valor de la propiedad respectiva falle en la validación.

Algunas anotaciones adicionales que se pueden encontrar en JSR:

@Pattern plantea que un campo de cadena solo es válido cuando coincide con una determinada expresión regular.

@Positive y @PositiveOrZero se aplican a valores numéricos y validan que sean estrictamente positivos, o positivos incluyendo 0.

@Negative y @NegativeOrZero se aplican a valores numéricos y validan que sean estrictamente negativos, o negativos incluyendo 0.

@Past y @PastOrPresent validan que un valor de fecha está en el futuro, o en el futuro, incluido el presente.

@Future y @FutureOrPresent validan que un valor de fecha está en el pasado o en el pasado, incluido el presente.

- ❖ Podemos informarle a Spring que queremos validar un determinado objeto mediante las anotaciones @Validated y @Valid.
- ❖ La @Validated es una anotación a nivel de clase que podemos usar para decirle a Spring que valide los parámetros que se pasan a un método de la clase anotada.
- ❖ La @Valid anotación a utilizar en los parámetros del método, podemos usarla para decirle a Spring que queremos que se valide un parámetro del método.

Uso de la anotación `@Valid` para validar el cuerpo de la solicitud

Para validar el cuerpo de la solicitud de una solicitud HTTP entrante, anotamos el cuerpo de la solicitud con la `@Valid` anotación en un controlador REST.

```
@PostMapping("/usuarios")
public ResponseEntity<?> create(@Valid @RequestBody Usuario cliente) {
    Usuario objUsuario = null;
    HashMap<String, Object> respuestas= new HashMap();
    ResponseEntity<?> objRespuesta;
    try
    {
        objUsuario = UsuarioService.save(cliente);
        objRespuesta= new ResponseEntity<Usuario>(objUsuario,HttpStatus.CREATED);
    }
    catch(DataAccessException e)
    {
        respuestas.put("mensaje", "Error al realizar la inserción en la base de datos");
        respuestas.put("descripción del error", e.getMessage());
        objRespuesta= new ResponseEntity<HashMap<String, Object>>(respuestas,HttpStatus.BAD_REQUEST);
    }

    return objRespuesta;
}
```

- En Spring MVC, tenemos muchas formas de establecer el código de estado de una respuesta HTTP. Una forma es usando la anotación `@ResponseStatus`.
- Para capturar excepciones utilizamos una solución a nivel `@Controller`. Definiremos un método para manejar excepciones y `anotarlo con @ExceptionHandler`:
- Solo está activo para ese controlador en particular, no globalmente para toda la aplicación. Por supuesto, agregar esto a cada controlador hace que no sea adecuado para un mecanismo general de manejo de excepciones.

Analizando el ejemplo del servicio que permite crear un usuario, retorna un código 201 o 400

Si la validación falla, activará una excepción `MethodArgumentNotValidException`. De forma predeterminada, Spring traducirá esta excepción a un estado HTTP 400 (Bad Request).

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
@ExceptionHandler(MethodArgumentNotValidException.class)
public Map<String, String> handleValidationExceptions(MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getAllErrors().forEach((error) -> {
        String fieldName = ((FieldError) error).getField();
        String errorMessage = error.getDefaultMessage();
        errors.put(fieldName, errorMessage);
    });
    return errors;
}
```

La anotación `@ExceptionHandler` nos permite manejar tipos específicos de excepciones a través de un solo método. Por tanto, podemos utilizarlo para procesar los errores de validación:

Validación del cuerpo de la petición

UNIVERSIDAD DEL CAUCA – FIET
DEPARTAMENTO DE SISTEMAS

Prueba

The screenshot shows a REST client interface with a POST request to `localhost:8080/api/usuarios`. The request body is a JSON object with the following fields:

```
1 {  
2   "username": "",  
3   "email": "adriana@",  
4   "fechaRegistro": "2025-08-09T05:00:00.000+00:00",  
5   "telefono": "799999999",  
6   "salario": -5000000  
7 }
```

The response status is **400 Bad Request** with a time of 208 ms and a size of 444 B. The response body contains validation errors for the fields:

```
1 {  
2   "fechaRegistro": "tiene que ser una fecha en el pasado o en el presente",  
3   "salario": "tiene que ser mayor o igual a 0",  
4   "telefono": "tiene que corresponder a la expresión regular \"[6][0-9]{8}\"",  
5   "email": "no es una dirección de correo bien formada",  
6   "username": "el tamaño tiene que estar entre 5 y 45"  
7 }
```

A red rectangle highlights the validation error messages in the response body.

Validación del cuerpo de la petición

UNIVERSIDAD DEL CAUCA – FIET
DEPARTAMENTO DE SISTEMAS

Prueba

The screenshot displays a REST client interface with a POST request to `localhost:8080/api/usuarios`. The request body is a JSON object with the following fields: `username`, `email`, `fechaRegistro`, `telefono`, and `salario`. The response status is 201 Created, and the response body is a JSON object with the same fields, plus an `id` field. A red rounded rectangle highlights the response body.

POST localhost:8080/api/usuarios

Untitled Request

POST localhost:8080/api/usuarios

Send Save

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Code

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "username": "adriana",
3   "email": "adriana@unicauca.edu.co",
4   "fechaRegistro": "2021-08-09T05:00:00.000+00:00",
5   "telefono": "699999999",
6   "salario": 5000000
7 }
```

Body Cookies Headers (5) Test Results

Status: 201 Created Time: 125 ms Size: 323 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 5,
3   "username": "adriana",
4   "email": "adriana@unicauca.edu.co",
5   "fechaRegistro": "2021-08-09T05:00:00.000+00:00",
6   "telefono": "699999999",
7   "salario": 5000000.0
8 }
```

La validación de variables de ruta y parámetros de solicitud funciona de manera un poco diferente.

```
@RestController
@RequestMapping("/api")
@Validated
public class UsuarioRestController {

    @Autowired
    private IUuariosServices UsuarioService;

    @GetMapping("/usuarios")
    public List<Usuario> index() {
        return UsuarioService.findAll();
    }
}
```

```
@GetMapping("/usuarios/{id}")
public ResponseEntity<?> show(@PathVariable @Min(5) Integer id) {
    Usuario objUsuario = null;
    HashMap<String, Object> respuestas= new HashMap();
    ResponseEntity<?> objRespuesta;
    try{
        objUsuario = UsuarioService.findById(id);

        if(objUsuario==null)
        {
            respuestas.put("mensaje", "El usuario con ID: "+id+" no existe en la base de datos");
            objRespuesta= new ResponseEntity<HashMap<String, Object>>(respuestas,HttpStatus.NOT_FOUND);
        }
        else
        {
            objRespuesta= new ResponseEntity<Usuario>(objUsuario,HttpStatus.OK);
        }
    }
    catch(DataAccessException e)
    {
        respuestas.put("mensaje", "Error al realizar la consulta en la base de datos");
        respuestas.put("descripción del error", e.getMessage());
        objRespuesta= new ResponseEntity<HashMap<String, Object>>(respuestas,HttpStatus.INTERNAL_SERVER_ERROR);
    }

    return objRespuesta;
}
```


La validación de variables de ruta y parámetros de solicitud funciona de manera un poco diferente.

No estamos validando objetos complejos de Java en este caso, ya que las variables de ruta y los parámetros de solicitud son tipos primitivos como `int` o sus objetos homólogos como `Integer` o `String`.

En lugar de anotar un campo de clase como el anterior, estamos agregando una anotación de restricción (en este caso `@Min`) directamente al parámetro del método en el controlador Spring:

Tenemos que agregar la `@Validated` anotación de Spring al controlador a nivel de clase para decirle a Spring que evalúe las anotaciones de restricción en los parámetros del método.

A diferencia de la validación del cuerpo de la solicitud, una validación fallida activará `ConstraintViolationException`.

Spring no registra un controlador de excepción predeterminado para esta excepción, por lo que de manera predeterminada causará una respuesta con el estado HTTP 500 (Error interno del servidor).

Si queremos devolver un estado HTTP 400 en su lugar (lo cual tiene sentido, ya que el cliente proporcionó un parámetro no válido, lo que lo convierte en una solicitud incorrecta), podemos agregar un método de excepción personalizado a nuestro controlador:

```
@ExceptionHandler({ConstraintViolationException.class})
@ResponseStatus(HttpStatus.BAD_REQUEST)
ResponseBody<String> handleConstraintViolationException(ConstraintViolationException e) {
    return new ResponseEntity<>("nombre del método y parametros erroneos: " + e.getMessage(), HttpStatus.BAD_REQUEST);
}
```

Prueba

The screenshot shows a REST client interface with the following components:

- Request Bar:** Method: GET, URL: localhost:8080/api/usuarios/1. Buttons: Send, Save.
- Request Tabs:** Params, Authorization, Headers (6), **Body** (selected), Pre-request Script, Tests, Settings. Additional links: Cookies, Code.
- Body Format:** none, form-data, x-www-form-urlencoded, **raw** (selected), binary, GraphQL, JSON. A Beautify button is also present.
- Request Body:** A large empty text area with a line number '1' at the top left.
- Response Section:**
 - Response Tabs:** **Body** (selected), Cookies, Headers (4), Test Results.
 - Status Bar:** Status: 400 Bad Request, Time: 992 ms, Size: 228 B. A Save Response button is on the right.
 - Response Format:** Pretty, Raw, Preview, Visualize. A dropdown menu is set to 'Text'.
 - Response Content:** A text area containing the message: `1 nombre del método y parametros erroneos: show.id: tiene que ser mayor o igual que 5`.

Por defecto las anotaciones proporcionan un mensaje de error genérico y descriptivo, pero generalmente vamos a querer definir nuestros propios mensajes de error personalizados. Esto podemos hacerlo utilizando el atributo *message* de las anotaciones.

```
@Entity
@Table(name = "Usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @NotNull message = "el nombre del usuario es obligatorio")
    @Size(min = 5, max = 45, message="la cantidad de caracteres del nombre debe estar entre 5 y 45")
    @Column(nullable = false, length = 45)
    private String username;
    @NotNull
    @Email
    private String email;
    @PastOrPresent
    private Date fechaRegistro;
    @Pattern(regexp = "[6][0-9]{8}")
    private String telefono;
    @PositiveOrZero
    private float salario;
```

Agregar mensajes a las validaciones

Prueba

Nos retorna el servidor el mensaje personalizado

The screenshot displays a REST client interface with the following components:

- Request Section:**
 - Method: POST
 - URL: localhost:8080/api/usuarios/
 - Buttons: Send, Save
- Request Body (JSON):**

```
{  "username": "",  "email": "andrea@unicauca.edu.co",  "fechaRegistro": "2021-08-09T05:00:00.000+00:00",  "telefono": "699999999",  "salario": 8000000.0}
```
- Response Section:**
 - Status: 509 Bandwidth Limit Exceeded (Apache bw/limited extensi...)
 - Time: 1177 ms, Size: 291 B
 - Buttons: Save Response
- Response Body (JSON):**

```
{  "username": "la cantidad de caracteres del nombre debe estar entre 5 y 45"}
```

Agregar mensajes personalizados a las validaciones

Definir el mensaje que verá el usuario tal cual es una mala práctica ya que habitualmente tendremos que localizar nuestras aplicaciones en varios idiomas.

Además, tenemos los mensajes codificados dentro del código fuente lo que obliga a recompilarlo ante cualquier cambio en los mismos.

Por ello se debe utilizar una clave que permita identificar el mensaje de error a mostrar, y obtener el texto de ese mensaje en el idioma adecuado a partir de ficheros de localización.

```
@Entity
@Table(name = "Usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @NotNull(message = "{user.name.empty}")
    @Size(min = 5, max = 45, message="la cantidad de caracteres del nombre debe estar entre 5 y 45")
    @Column(nullable = false, length = 45)
    private String username;

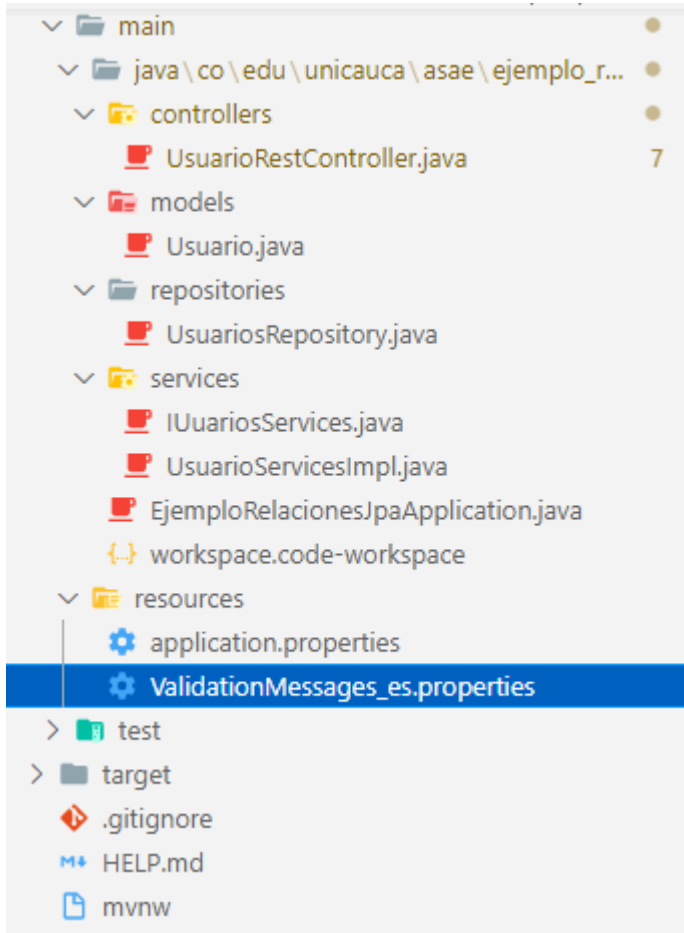
    @NotNull(message = "{user.email.empty}")
    @Email(message = "{user.email.mask}")
    private String email;

    @PastOrPresent(message = "{user.date.past}")
    private Date fechaRegistro;

    @Pattern(message = "{user.telephone.pattern}", regex = "[6][0-9]{8}")
    private String telefono;

    @PositiveOrZero (message = "{user.salary.positive}")
    private float salario;
}
```

Agregar mensajes personalizados a las validaciones



En el classpath de la aplicación (/src/main/resources) creamos para cada idioma ficheros .properties. El fichero con los textos por defecto será ValidationMessages.properties, mientras que para cada idioma seguimos la convención de crear un fichero con el nombre «ValidationMessages» seguido del código de localización según la respuesta de la llamada a Locale.getDefault(). Las tildes van en Unicode.

```
ValidationMessages_es.properties X
src > main > resources > ValidationMessages_es.properties
1  user.name.empty = El nombre est\u00E1 vac\u00EDo
2  user.email.mask= Correo no valido
3  user.email.empty= El correo est\u00E1 vac\u00EDo
4  user.date.past=La fecha no puede estar en el futuro
5  user.telephone.pattern= El telefono debe empezar con el número 6 y tener 9 digitos
6  user.salary.positive= El salario no puede ser negativo
```

Agregar mensajes personalizados a las validaciones

Prueba

Los mensajes de los errores se resuelven en función del locale que tenga configurado Java por defecto en el momento de ejecutarse la validación.

POST localhost:8080/api/usuarios/ Send Save

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Code

● none ● form-data ● x-www-form-urlencoded ● raw ● binary ● GraphQL JSON Beautify

```
1 {
2   "email": "andrea@",
3   "fechaRegistro": "2025-08-09T05:00:00.000+00:00",
4   "telefono": "799999999",
5   "salario": 8000000.0
6 }
```

Body Cookies Headers (5) Test Results Status: 509 Bandwidth Limit Exceeded (Apache bw/limited extension) Time: 3.15 s Size: 408 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "fechaRegistro": "La fecha no puede estar en el futuro",
3   "telefono": "El telefono debe empezar con el número 6 y tener 9 digitos",
4   "email": "Correo no valido",
5   "username": "El nombre está vacío"
6 }
```


Con **@Valid** se detecta si hay errores en la validación de los datos. Si hay errores el controlador va a retornar los errores y el código 400

```
@PostMapping("/clientes")
public ResponseEntity<?> create(@Valid @RequestBody Cliente cliente, BindingResult result) {
    cliente.setCreateAt(new Date());
    Map<String, Object> response = new HashMap<>();
    Cliente objCliente;

    if(result.hasErrors())
    {
        List<String> listaErrores= new ArrayList<>();

        for (FieldError error : result.getFieldErrors()) {
            listaErrores.add("Campo " + error.getField() + ": " + error.getDefaultMessage());
        }
        response.put("errors", listaErrores);
        return new ResponseEntity<Map<String, Object>>(response, HttpStatus.BAD_REQUEST);
    }

    try {
        objCliente=this.clienteService.save(cliente);
    }
    catch(DataAccessException e) {
        response.put("mensaje", "Error al realizar la inserción en la base de datos");
        response.put("error", e.getMessage() + "" + e.getMostSpecificCause().getMessage());
        return new ResponseEntity<Map<String, Object>>(response, HttpStatus.INTERNAL_SERVER_ERROR);
    }

    return new ResponseEntity<Cliente>(objCliente, HttpStatus.OK);
}
```

BindingResult

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: localhost:8085/api/clientes/
 - Body (JSON):

```
{  1  {  2    "nombre": "juan",  3    "apellido": "perez",  4    "email": "juan@unicauca.edu.co",  5    "fechaRegistro": "2021-08-09T05:00:00.000+00:00"  6  }
```
- Response:**
 - Status: 200 OK (highlighted with a red box)
 - Time: 314 ms
 - Size: 370 B
 - Save Response (dropdown arrow)
- Response Body (Pretty View, highlighted with a red box):**

```
1  {  2    "id": 3,  3    "nombre": "juan",  4    "apellido": "perez",  5    "email": "juan@unicauca.edu.co",  6    "createAt": "2022-02-04T15:22:38.509+00:00"  7  }
```

BindingResult

POST localhost:8085/api/clientes/ Send Save

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies Code

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON Beautify

```
1 {
2   "nombre": "jua",
3   "apellido": "per",
4   "email": "juane@unicauca.edu.co",
5   "fechaRegistro": "2022-08-09T05:00:00.000+00:00"
6 }
```

Body Cookies Headers (7) Test Results Status: 400 Bad Request Time: 63 ms Size: 413 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "errors": [
3     "Campo 'nombre: ' la cantidad de caracteres del nombre debe estar entre 4 y 12",
4     "Campo 'apellido: ' la cantidad de caracteres del apellido debe estar entre 4 y 12"
5   ]
6 }
```

<https://docs.jboss.org/hibernate/beanvalidation/spec/2.0/api/javax/validation/constraints/package-summary.html>

<https://reflectoring.io/bean-validation-with-spring-boot/>

<https://danielme.com/2018/10/10/validaciones-con-hibernate-validator/>

Muchas gracias
Preguntas

